

使用 Antigravity 建構及部署至 Google Cloud

來源：<https://codelabs.developers.google.com/build-and-deploy-gcp-with-antigravity?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將學習如何使用 Google Antigravity 設計、建構及部署無伺服器應用程式至 Google Cloud。

目錄

1. [1. 簡介](#)
2. [2. 應用程式總覽](#)
3. [3. 規劃架構](#)
4. [4. 生成應用程式](#)
5. [5. 驗證應用程式](#)
6. [6. 瞭解應用程式](#)
7. [7. 擴展應用程式](#)
8. [8. 結論](#)

步驟 1 · 約 5 分鐘

1. 簡介

從做中學 免付費

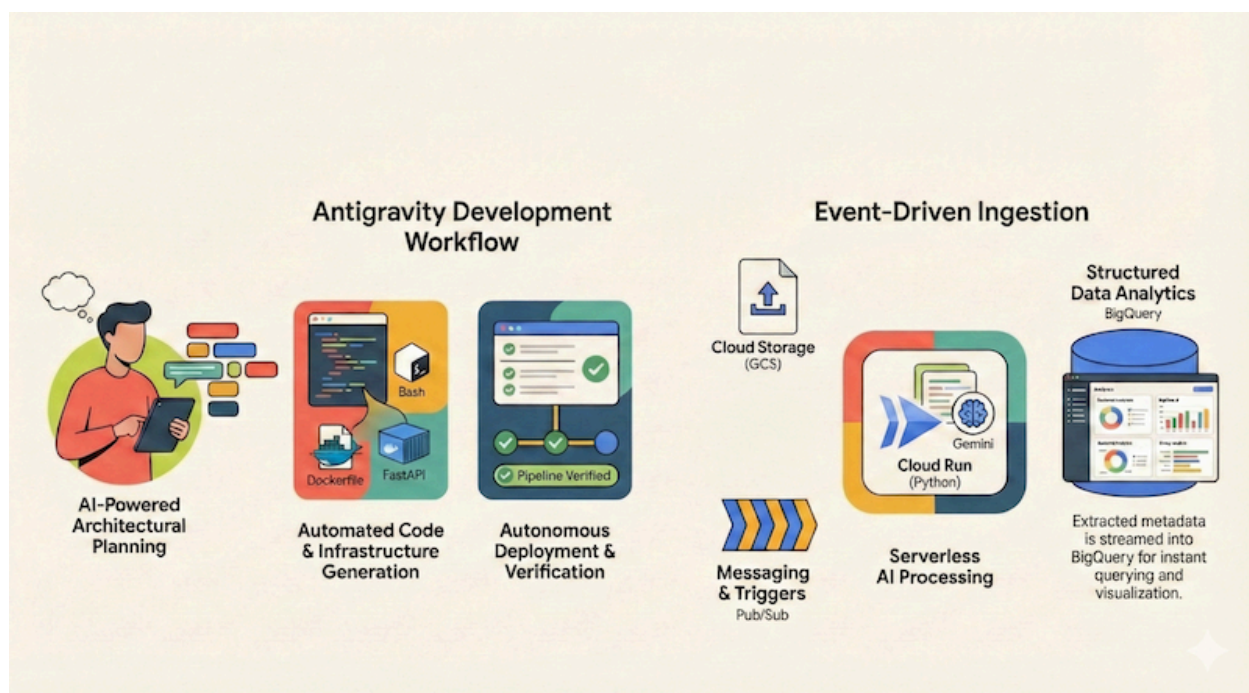
立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用



在本程式碼研究室中，您將瞭解如何使用 [Google Antigravity](#) 設計、建構及部署無伺服器應用程式至 Google Cloud。我們將建構無伺服器且事件驅動的文件管道，從 Google Cloud Storage (GCS) 擷取檔案、使用 Cloud Run 和 Gemini 處理檔案，並將中繼資料串流至 BigQuery。

課程內容

- 如何使用 Antigravity 進行架構規劃和設計。
- 使用 AI 代理生成基礎架構即程式碼 (Shell 指令碼)。
- 建構及部署以 Python 為基礎的 Cloud Run 服務。
- 整合 Vertex AI 的 Gemini，進行多模態文件分析。
- 使用 Antigravity 的 Walkthrough 構件驗證端對端管道。

軟硬體需求

- 已安裝 [Google Antigravity](#)。如需安裝 Antigravity 的相關協助，以及瞭解基本概念，建議完成「[開始使用 Google Antigravity](#)」程式碼研究室。
- 已啟用計費功能的 Google Cloud 專案。

- 已安裝並驗證 [gcloud CLI](#)。
-

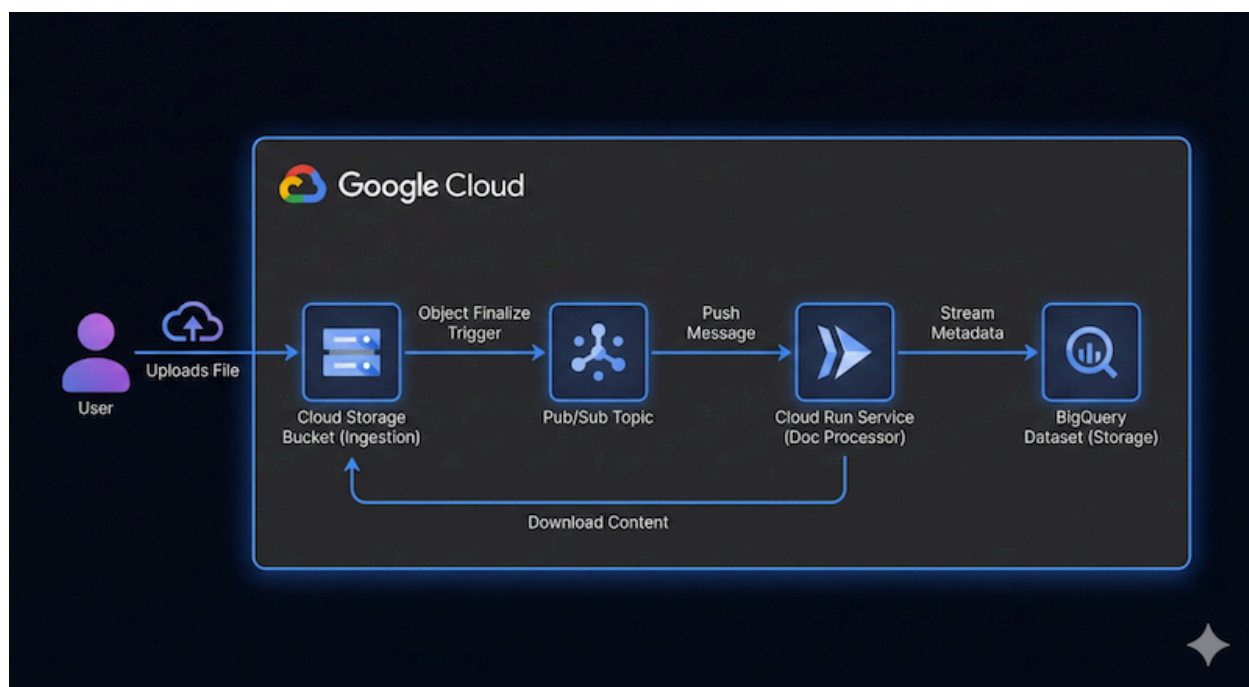
步驟 2 · 約 2 分鐘

2. 應用程式總覽

在開始使用 Antigravity 架構及實作應用程式之前，我們先來規劃要建構的應用程式。

我們想建構無伺服器且事件驅動的文件管道，從 Google Cloud Storage (GCS) 擷取檔案、使用 Cloud Run 和 Gemini 處理檔案，並將中繼資料串流至 BigQuery。

這個應用程式的大致架構圖可能如下所示：



這項資訊不必精確，Antigravity 可協助我們逐步瞭解架構詳細資料。不過，最好先瞭解自己想建構什麼。您提供的詳細資料越多，Antigravity 就能產生越符合需求的架構和程式碼。

步驟 3 · 約 10 分鐘

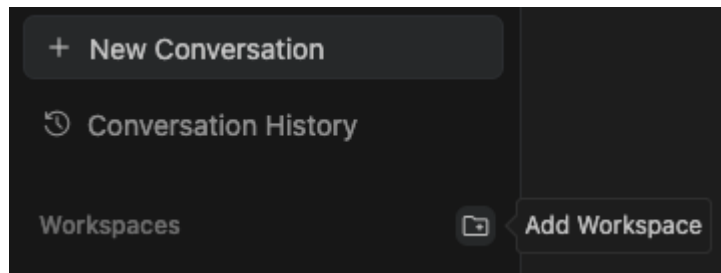
3. 規劃架構

我們已準備好使用 Antigravity 開始規劃架構詳細資料！

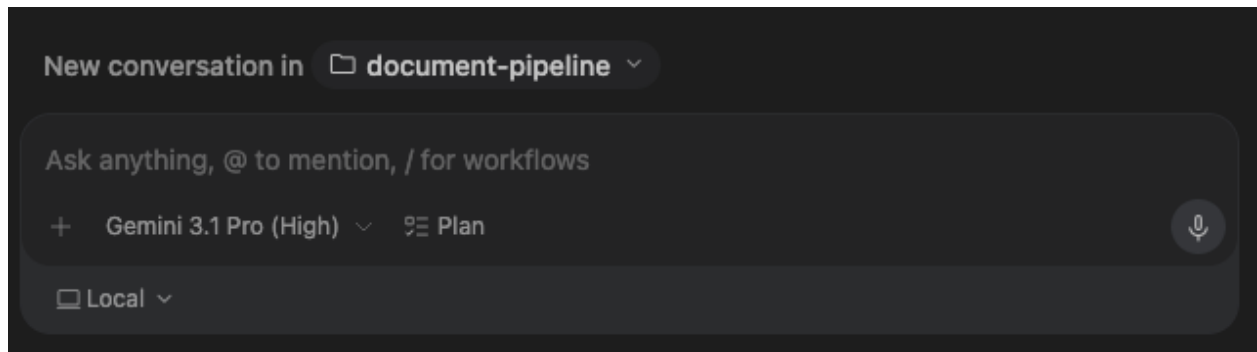
Antigravity 擅長規劃複雜系統，我們可以先定義高階架構，而不是立即編寫程式碼。

首先，請確認您位於代理程式管理員中。如果剛開啟「反重力」模式，可以點選中間的 `Open Agent Manager` 按鈕，或右上角也會顯示相同的 `Open Agent Manager`。

首先，請為應用程式建立資料夾 (例如 `document-pipeline`)，然後在 Agent Manager 中將該資料夾新增為工作區：



在代理程式管理員中，於 `document-pipeline` 工作區發起對話。確認「規劃」模式也已啟用：



按一下右上角的設定圖示 ⚙️，然後將 `Review Policy` 下方的 `Artifact` 設為 `Asks for Review`，並將 `Terminal Command Auto Execution` 下方的 `Terminal` 設為 `Request Review`。這樣一來，您就能在每個步驟中審查並核准計畫，再由專員執行。

提示詞

現在，我們準備為 Antigravity 提供第一個提示。

首先，請確認 Antigravity 處於 `Plan` 模式，並選擇 `Gemini Pro (High)` 模型 (您也可以嘗試其他模型)。

輸入下列提示詞，然後按一下「提交」按鈕：

```
I want to build a serverless event-driven document processing pipeline on Google Cloud.
Architecture:
- Ingestion: Users upload files to a Cloud Storage bucket.
- Trigger: File uploads trigger a Pub/Sub message.
- Processor: A Python-based Cloud Run service receives the message, processes the file
(simulated OCR), and extracts metadata.
- Storage: Stream the metadata (filename, date, tags, word_count) into a BigQuery dataset.
```

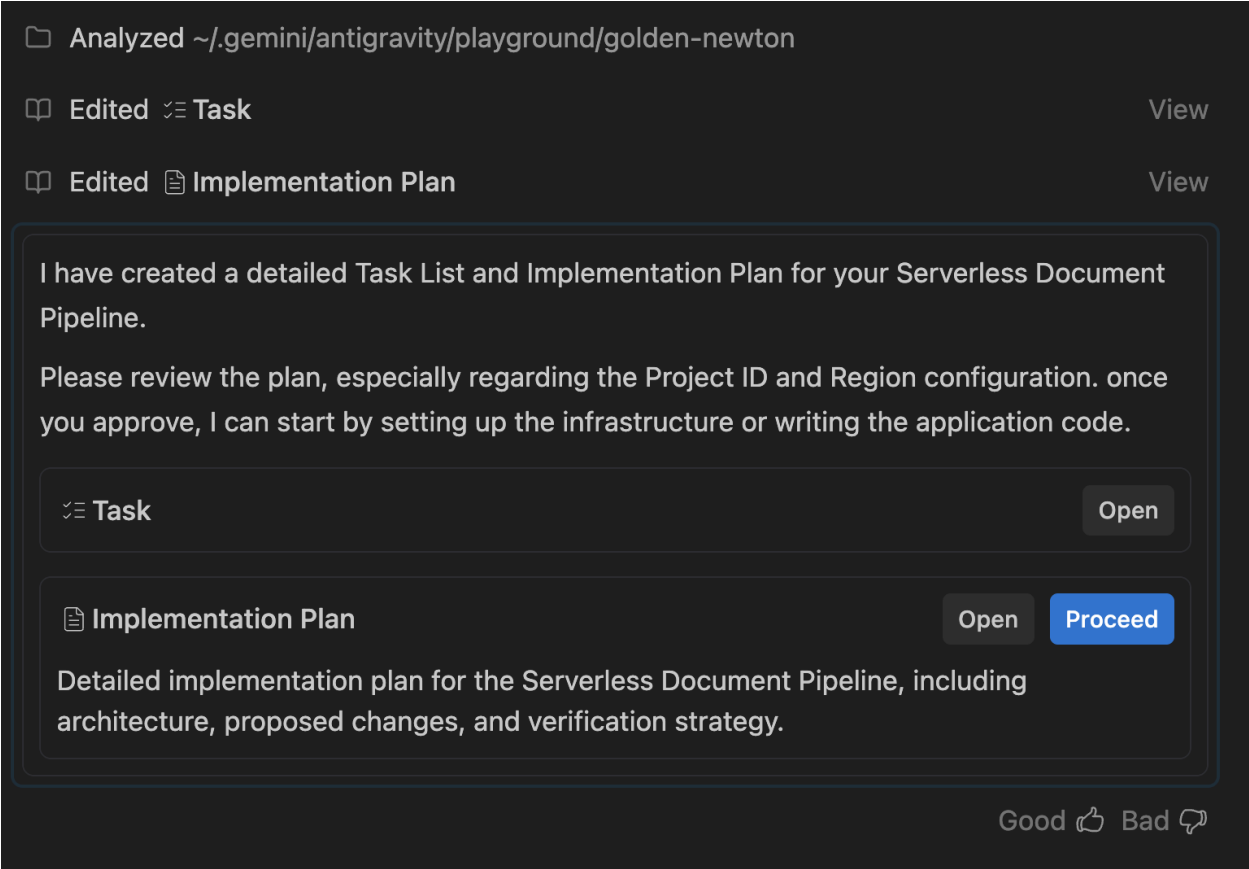
工作清單和導入計畫

Antigravity 會分析您的要求，然後生成「工作」清單和「實作計畫」。

這項計畫涵蓋：

- **基礎架構**：GCS Bucket、Pub/Sub 主題、BigQuery 資料集。
- **處理器**：Python/Flask 應用程式、Dockerfile、需求。
- **整合**：GCS 通知 → Pub/Sub → Cloud Run。

您應該會看到類似下列的輸出：



按一下「Task」列旁的「Open」按鈕。這時應該會顯示 Antigravity 建立的一系列工作。專員會逐一檢查：

Serverless Document Pipeline Task List

☐ Infrastructure Setup

- ☐ Create Google Cloud Project (or use existing)
- ☐ Enable necessary APIs (Storage, Pub/Sub, Cloud Run, BigQuery, Artifact Registry)
- ☐ Set up IAM roles and service accounts

☐ Ingestion Layer (Cloud Storage)

- ☐ Create GCS bucket for uploads
- ☐ Configure Pub/Sub notifications for object finalization

☐ Messaging Layer (Pub/Sub)

- ☐ Create Pub/Sub topic
- ☐ Create Push subscription to Cloud Run service (endpoint TBA)

☐ Data Storage Layer (BigQuery)

- ☐ Create BigQuery dataset
- ☐ Define and create the table schema (filename, date, tags, word_count)

☐ processing Layer (Cloud Run)

- ☐ Scaffold Python application (FastAPI or Flask)
- ☐ Implement GCS file download
- ☐ Implement simulated OCR / metadata extraction logic
- ☐ Implement BigQuery streaming insert
- ☐ Dockerize the application
- ☐ Deploy to Cloud Run

☐ Verification & Testing

- ☐ Test local execution (using emulator or mocked events)
- ☐ End-to-end test: Upload file -> Check BigQuery

下一步是查看實作計畫，並授權給代理商繼續進行。

按一下實作計畫即可查看詳細資料。請仔細閱讀。請把握這個難得的機會，針對實作作業提供意見。您可以點選導入計畫的任何部分，然後新增註解。新增一些註解後，請務必提交您想看到的變更項目以供審查，特別是命名、Google Cloud 雲端專案 ID、區域等。

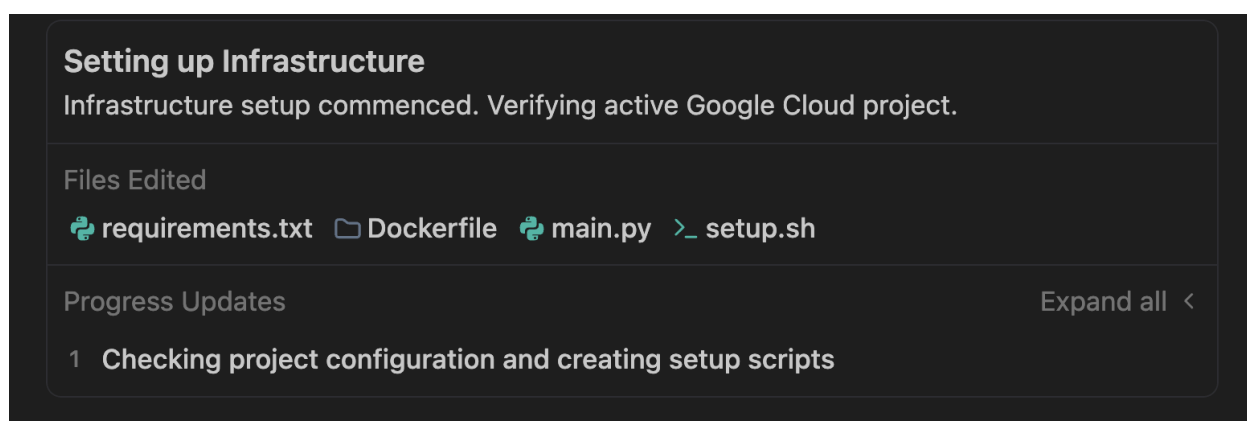
確認一切無誤後，按一下 **Proceed** 按鈕，授權代理人繼續執行導入計畫。

4. 生成應用程式

計畫獲准後，Antigravity 就會開始生成應用程式所需的檔案，包括佈建指令碼和應用程式程式碼。

注意：系統會提示多個終端機指令 (gcloud、殼層指令碼等)，代理程式需要使用這些指令瞭解環境，然後據此產生指令碼。按一下 **Accept** 按鈕，讓代理程式向前移動。本實驗室會介紹產生的特定檔案名稱，但您可能會發現 Antigravity 產生不同的檔案名稱，不過檔案的用途和內容應該類似。

Antigravity 會建立資料夾，並開始建立專案所需的檔案。在我們的範例執行中，我們看到以下內容：

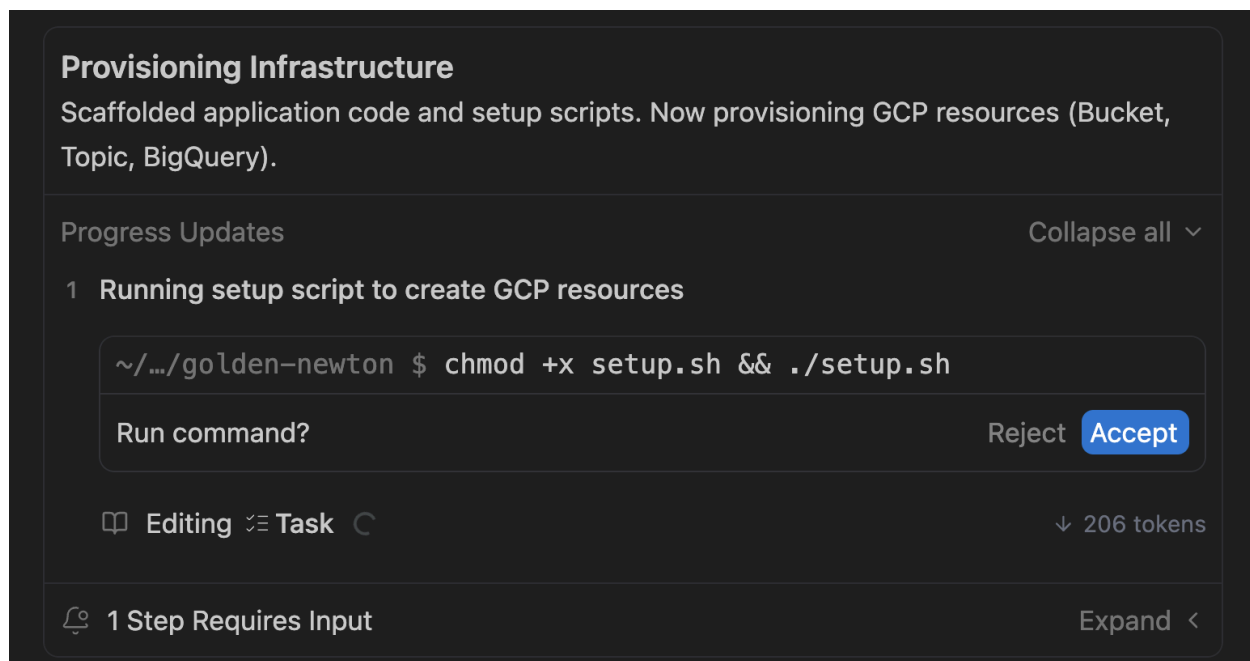


系統會產生 `setup.sh` 或類似名稱的殼層指令碼檔案，自動建立資源。可處理：

1. 啟用 API (`run` 、 `pubsub` 、 `bigquery` 、 `storage`)。
2. 建立 Google Cloud Storage bucket (`doc-ingestion-{project-id}`)。
3. 建立 BigQuery 資料集和資料表 (`pipeline_data.processed_docs`)。
4. 設定 Pub/Sub 主題和通知。

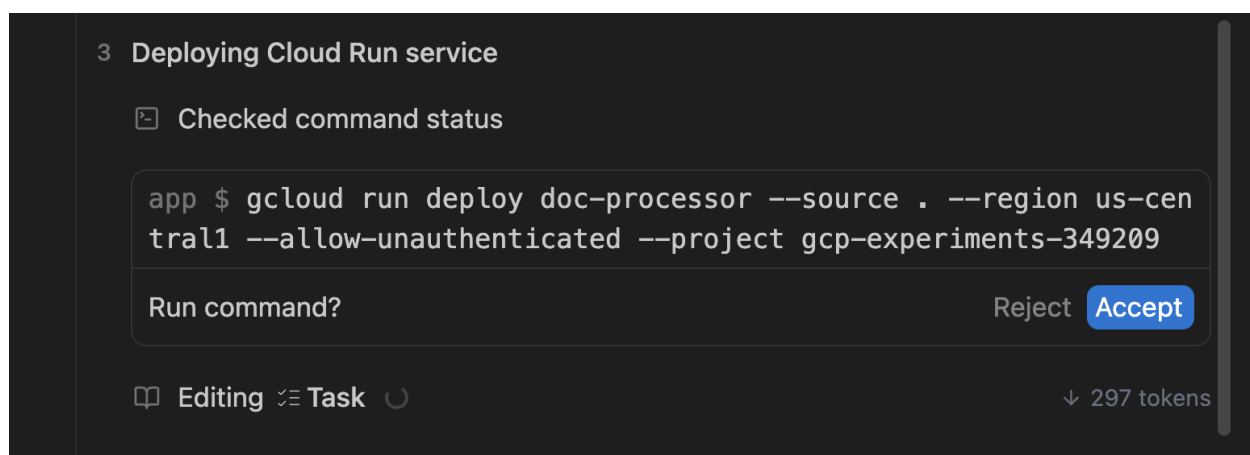
代理程式應產生 Python 應用程式 (`main.py`)，監聽 Pub/Sub 推送訊息。這項服務會使用模擬的 OCR 邏輯，產生隨機字數和標記，並將這些資料保留在 BigQuery 中。

產生這項設定指令碼後，Antigravity 應會提示您代表您執行指令碼。畫面範例如下所示：



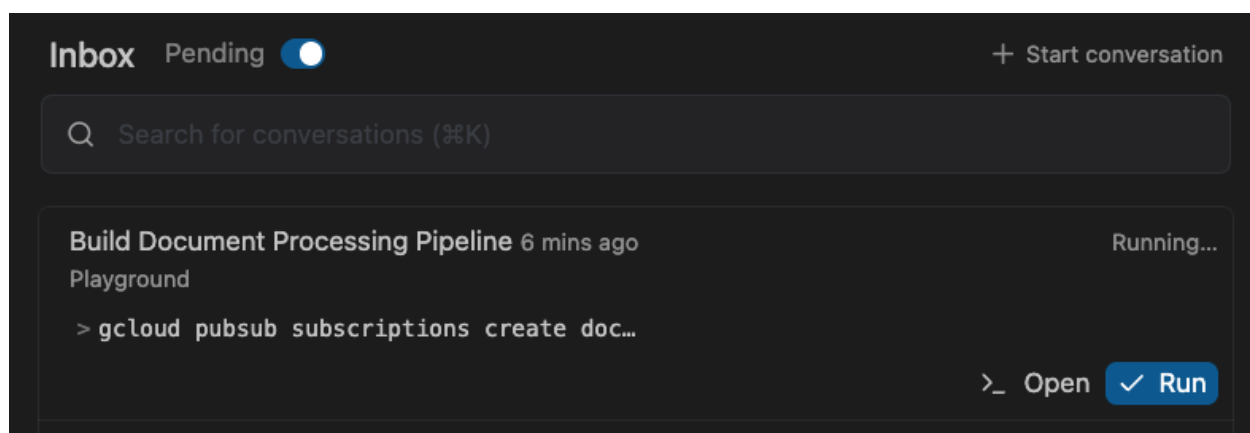
視需要點選 `Accept`。

佈建指令碼會建立資源，並驗證資源是否已建立。檢查成功後，系統會繼續建構容器，並以服務的形式部署至 Cloud Run (位於 `main.py`)。輸出範例如下所示：



部署服務時，系統也會設定 Pub/Sub 訂閱項目和其他膠合，確保管道正常運作。這一切應該只需要幾分鐘。

在此期間，您可以切換至 `Inbox` (位於左上角)，查看等待您輸入的 `Pending` 工作：



這樣一來，您就能確保在代理程式尋求意見回饋時核准工作。

5. 驗證應用程式

管道部署完成後，Antigravity 會繼續驗證應用程式是否正常運作。Antigravity 會自動執行這項操作，並在工作清單中加入驗證步驟。畫面範例如下所示：

Verifying Pipeline

Infrastructure provisioned and configured. Cloud Run service deployed. Testing end-to-end flow.

Progress Updates

Collapse all ▾

1 Uploading test document and querying BigQuery

```
~/.../golden-newton $ echo "This is a test document with some words to count." > test.txt && gsutil cp test.txt gs://gcp-experiments-349209-doc-uploads/
```

Run command?

Reject

Accept

這個指令會建立測試構件 (`test.txt`)，並將其上傳至 Google Cloud Storage bucket。按一下 `Accept` 繼續操作。

若要自行執行進一步測試，可以試著將範例檔案上傳至 Cloud Storage bucket：

```
gsutil cp <some-test-doc>.txt gs://doc-ingestion-{project-id}/
```

在 BigQuery 中查看結果

驗證程序也會檢查資料是否已保留在 BigQuery 中。

Progress Updates

Collapse all ▾

1 Uploading test document and querying BigQuery

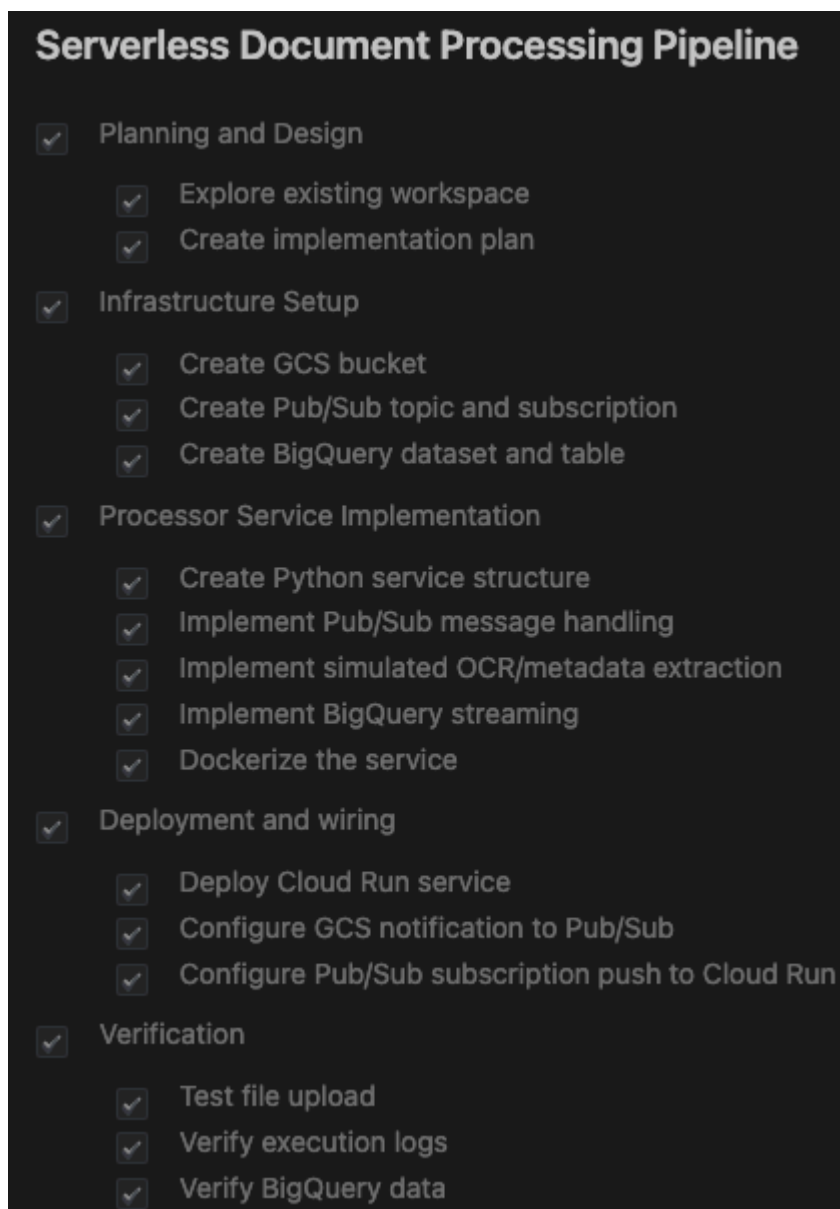
☒ Checked command status

```
~/.../golden-newton $ sleep 10 && bq query --nouse_legacy_sql 'SELECT * FROM `gcp-experiments-349209.doc_pipeline.processed_docs` LIMIT 10'
```

filename	upload_date	tags	word_count
test.txt	2026-01-29 05:26:04	scanned,reviewed	10

請注意，系統用來檢查文件的 SQL 查詢。

驗證完成後，您應該會看到工作清單已完成：



選用：手動驗證

即使 Antigravity 已驗證應用程式，您也可以視需要按照下列步驟，在 Google Cloud 控制台中手動檢查是否已建立所有資源。

Cloud Storage

目標：確認值區存在，並檢查是否有上傳的檔案。

1. 依序前往「Cloud Storage」>「Buckets」。
2. 找出名為 `[PROJECT_ID]-doc-uploads` 的 bucket。
3. 按一下 bucket 名稱即可瀏覽檔案。
4. 確認：您應該會看到上傳的檔案 (例如 `test.txt`)。

Pub/Sub

目標：確認主題存在，且已有推送訂閱項目。

1. 依序前往「Pub/Sub」>「主題」。
2. 找到 **doc-processing-topic**。
3. 按一下**主題 ID**。
4. 向下捲動至「訂閱項目」分頁標籤。
5. 驗證：確認「doc-processing-sub」列於清單中，且傳送類型為「Push」。

Cloud Run

目標：檢查服務狀態和記錄。

1. 前往 **Cloud Run**。
2. 按一下「doc-processor」服務。
3. 驗證：
4. 健康狀態：綠色勾號表示服務已啟用。
5. 記錄：按一下「記錄」分頁標籤。尋找「**Processing file: gs://...**」和「**Successfully processed...**」等項目。

BigQuery

目標：驗證資料是否確實已儲存。

1. 前往「BigQuery」>「SQL 工作區」。
2. 在「Explorer」窗格中，依序展開「專案」>「pipeline_data 資料集」。
3. 點選「processed_docs」**processed_docs**資料表。
4. 點選「預覽」分頁標籤。
5. 確認：您應該會看到包含 **filename**、**upload_date**、**tags** 和 **word_count** 的資料列。

逐步操作說明

最後，Antigravity 會生成逐步操作說明構件。這項成果會摘要說明：

- 已完成的變更。
- 執行驗證指令。
- 實際結果 (查詢輸出內容，顯示 Gemini 擷取的中繼資料)。

按一下 **Open** 即可查看。輸出範例如下所示：

Serverless Document Pipeline Walkthrough

I have successfully built and deployed the serverless document pipeline.

Architecture Verified

1. **Ingestion:** `test.txt` uploaded to `gs://gcp-experiments-349209-doc-uploads`
2. **Trigger:** Pub/Sub notification sent to `doc-processing-topic`
3. **Processing:** Cloud Run service `doc-processor` triggered via Push Subscription
4. **Storage:** Metadata inserted into BigQuery table `doc_pipeline.processed_docs`

Verification Steps

1. File Upload

Uploaded a test file with sample text.

```
echo "This is a test document with some words to count." > test.txt
gsutil cp test.txt gs://gcp-experiments-349209-doc-uploads/
```

2. BigQuery Validation

Queried the BigQuery table to confirm processing.

```
bq query --nouse_legacy_sql 'SELECT * FROM `gcp-experiments-349209.doc_pipeline.processed_docs` LIMIT 10'
```

Result:

filename	upload_date	tags	word_count
test.txt	2026-01-29 05:26:04	scanned,reviewed	10

The pipeline is fully operational.

6. 瞭解應用程式

此時，您已佈建並執行基本應用程式。在進一步擴充這個應用程式之前，請先花點時間探索程式碼。按一下右上角的 `Open Editor` 按鈕，即可切換至編輯器。

以下簡要說明您可能會看到的檔案：

1. `setup.sh`：主要指令碼，用於佈建所有 Google Cloud 資源，並啟用必要的 API。
2. `main.py`：管道的主要進入點。這個 Python 應用程式會建立網路伺服器，接收 Pub/Sub 推送訊息、從 GCS 下載檔案、「處理」檔案 (模擬 OCR)，並將中繼資料串流至 BigQuery。
3. `Dockerfile`：定義如何將應用程式封裝至容器映像檔。
4. `requirements.txt`：列出 Python 依附元件。

您也可能會看到測試和驗證所需的其他指令碼和文字檔。

7. 擴展應用程式

現在您已擁有可運作的基本應用程式，可以繼續疊代及擴充應用程式。以下提供幾個點子。

新增前端

建構簡單的網頁介面，以便查看處理過的文件。

請嘗試使用下列提示：`Create a simple Streamlit or Flask web application that connects to BigQuery. It should display a table of the processed documents (filename, upload_date, tags, word_count) and allow me to filter the results by tag`

與實際的 AI/ML 整合

請改用 Gemini 模型擷取、分類及翻譯，而非模擬 OCR 處理程序。

1. 取代虛擬 OCR 邏輯。將圖片/PDF 傳送給 Gemini，即可擷取實際文字和資料。分析擷取的文字，將文件分類 (例如帳單、合約、履歷)，或擷取實體 (例如日期、姓名、地點)。
2. 系統會自動偵測文件語言，並先翻譯成英文再儲存。你也可以使用其他語言。

提升儲存空間和數據分析功能

您可以在 bucket 中設定生命週期規則，將舊檔案移至「Coldline」或「Archive」儲存空間，以節省費用。

穩定性與安全性

您可以採取下列措施，讓應用程式更穩定可靠且安全：

1. 無效信件佇列 (DLQ)：更新 Pub/Sub 訂閱項目，處理失敗情況。如果 Cloud Run 服務無法處理檔案 5 次，請將訊息傳送至另一個「Dead Letter」主題/bucket，供人工檢查。
2. Secret Manager：如果應用程式需要 API 金鑰或機密設定，請將這些資訊儲存在 Secret Manager 中，並從 Cloud Run 安全地存取，而非將字串硬式編碼。
3. Eventarc：從直接使用 Pub/Sub 升級至 Eventarc，即可享有更彈性的事件轉送功能，根據複雜的稽核記錄或其他 GCP 服務事件觸發動作。

當然，你可以提出自己的想法，並使用 Antigravity 協助實作！

8. 結論

您已使用 Google Antigravity，在幾分鐘內成功建構可擴充的無伺服器 AI 輔助文件管道。您學到了下列內容：

- 運用 AI 規劃架構。
- 指示及管理 Antigravity，讓它從程式碼生成到部署和驗證，逐步生成應用程式。
- 透過逐步解說驗證部署作業。

參考文件

- 官方網站：<https://antigravity.google/>
 - 說明文件：<https://antigravity.google/docs>
 - 用途：<https://antigravity.google/use-cases>
 - 下載：<https://antigravity.google/download>
 - 程式碼研究室：[開始使用 Google Antigravity](#)
-